In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 120)
```

In [2]:
```
data = {
    "Order_ID": [f"ORD{1001+i}" for i in range(60)],
    "Customer_ID": [f"C{201+(i%25)}" for i in range(60)],
    "Customer_Name": [f"Customer {201+(i%25)}" for i in range(60)],
    "Product_ID": [f"P{(i%12)+1:03d}" for i in range(60)],
    "Product": ["Laptop", "Smartphone", "Headphones", "Smartwatch", "Tablet", "Keyboard",
                "Mouse", "Monitor", "Camera", "Speaker", "Charger", "Power Bank"] * 5,
    "Category": ["Electronics", "Electronics", "Accessories", "Wearables", "Electronics", "
                 "Accessories", "Electronics", "Electronics", "Audio", "Accessories", "Acce
    "Brand": ["Aster", "Nova", "SoundMax", "Pulse", "Aster", "KeyPro",
              "KeyPro", "ViewPoint", "Optix", "SoundMax", "Volt", "Volt"] * 5,
    "Quantity": [1,2,3,1,2,2,3,1,1,2,4,2] * 5,
    "Unit_Price": [65000, 28000, 2500, 6500, 22000, 1800, 900, 14500, 32000, 4200, 700, 160
    "Discount_Percent": [5,10,0,8,5,0,10,12,5,0,0,10] * 5,
    "Shipping_Cost": [500,150,60,80,180,50,40,250,300,90,30,50] * 5,
    "Payment_Method": ["UPI","Credit Card","Debit Card","UPI","Net Banking","UPI",
                       "Cash on Delivery","Credit Card","UPI","Debit Card","UPI","Net Banki
    "Order_Date": pd.date_range("2026-01-01", periods=60, freq="3D").strftime("%Y-%m-%d").t
    "Ship_Date": pd.date_range("2026-01-02", periods=60, freq="3D").strftime("%Y-%m-%d").to
    "Delivery_Status": ["Delivered","Delivered","Delivered","Shipped","Delivered","Cancelle
                        "Delivered","Returned","Delivered","Delivered","Shipped","Delivered
    "City": ["Hyderabad","Chennai","Bengaluru","Mumbai","Pune","Warangal"] * 10,
    "Region": ["South","South","South","West","West","South"] * 10,
    "Rating": [5,4,4,5,3, np.nan, 4,2,5,4,3,5] * 5,
    "Return_Status": ["No","No","No","No","No","No","No","Yes","No","No","No","No"] * 5
}
df = pd.DataFrame(data)
```

In [3]:
```
#Adding some issues on data like duplicates etc..
df.loc[3, "Customer_Name"] = "  CUSTOMER 204 "
df.loc[7, "Product"] = " laptop "
df.loc[10, "City"] = " chennai "
df.loc[12, "Quantity"] = "two"
df.loc[15, "Unit_Price"] = "unknown"
df.loc[18, "Order_Date"] = "not-a-date"
df.loc[22, "Customer_Name"] = None
df.loc[25, "Rating"] = np.nan
df.loc[31, "Payment_Method"] = None
df.loc[35, "Region"] = " south "
df.loc[40, "Shipping_Cost"] = np.nan
df.loc[45, "Delivery_Status"] = "delivered"
df.loc[50, "Discount_Percent"] = "five"
# for duplicates
df.loc[59, "Order_ID"] = df.loc[0, "Order_ID"]

df.head(10)
```

Out[3]:

	Order_ID	Customer_ID	Customer_Name	Product_ID	Product	Category	Brand	Quantity	Unit_Pric
0	ORD1001	C201	Customer 201	P001	Laptop	Electronics	Aster	1	6500
1	ORD1002	C202	Customer 202	P002	Smartphone	Electronics	Nova	2	2800
2	ORD1003	C203	Customer 203	P003	Headphones	Accessories	SoundMax	3	250
3	ORD1004	C204	CUSTOMER 204	P004	Smartwatch	Wearables	Pulse	1	650
4	ORD1005	C205	Customer 205	P005	Tablet	Electronics	Aster	2	2200
5	ORD1006	C206	Customer 206	P006	Keyboard	Accessories	KeyPro	2	180
6	ORD1007	C207	Customer 207	P007	Mouse	Accessories	KeyPro	3	90
7	ORD1008	C208	Customer 208	P008	laptop	Electronics	ViewPoint	1	1450
8	ORD1009	C209	Customer 209	P009	Camera	Electronics	Optix	1	3200
9	ORD1010	C210	Customer 210	P010	Speaker	Audio	SoundMax	2	420

In [4]:
```
df.shape
```

Out[4]:
```
(60, 19)
```

In [5]:
```
df.columns
```

Out[5]:
```
Index(['Order_ID', 'Customer_ID', 'Customer_Name', 'Product_ID', 'Product', 'Category', 'B
rand', 'Quantity',
       'Unit_Price', 'Discount_Percent', 'Shipping_Cost', 'Payment_Method', 'Order_Date',
'Ship_Date',
       'Delivery_Status', 'City', 'Region', 'Rating', 'Return_Status'],
      dtype='object')
```

In [6]:
```
df.dtypes
```

Out[6]:
```
Order_ID            object
Customer_ID         object
Customer_Name       object
Product_ID          object
Product             object
Category            object
Brand               object
Quantity            object
Unit_Price          object
Discount_Percent    object
Shipping_Cost      float64
Payment_Method      object
Order_Date          object
Ship_Date           object
Delivery_Status     object
City                object
Region              object
Rating             float64
Return_Status       object
dtype: object
```

In [7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 60 entries, 0 to 59
Data columns (total 19 columns):
 #   Column            Non-Null Count  Dtype  
---  ------            --------------  -----  
 0   Order_ID          60 non-null     object 
 1   Customer_ID       60 non-null     object 
 2   Customer_Name     59 non-null     object 
 3   Product_ID        60 non-null     object 
 4   Product           60 non-null     object 
 5   Category          60 non-null     object 
 6   Brand             60 non-null     object 
 7   Quantity          60 non-null     object 
 8   Unit_Price        60 non-null     object 
 9   Discount_Percent  60 non-null     object 
 10  Shipping_Cost     59 non-null     float64
 11  Payment_Method    59 non-null     object 
 12  Order_Date        60 non-null     object 
 13  Ship_Date         60 non-null     object 
 14  Delivery_Status   60 non-null     object 
 15  City              60 non-null     object 
 16  Region            60 non-null     object 
 17  Rating            54 non-null     float64
 18  Return_Status     60 non-null     object 
dtypes: float64(2), object(17)
memory usage: 9.0+ KB
```

In [8]:
```
df.describe()
```

Out[8]:

	Shipping_Cost	Rating
count	59.000000	54.000000
mean	147.796610	4.000000
std	137.327544	0.971286
min	30.000000	2.000000
25%	50.000000	3.000000
50%	80.000000	4.000000
75%	215.000000	5.000000
max	500.000000	5.000000

In [9]:
```
# Standardize text columns
text_columns = ["Customer_Name", "Product", "Category", "Brand", "Payment_Method",
                "Delivery_Status", "City", "Region", "Return_Status"]
for col in text_columns:
    df[col] = df[col].astype("string").str.strip()

df["Customer_Name"] = df["Customer_Name"].replace("", pd.NA)
df["Payment_Method"] = df["Payment_Method"].replace("", pd.NA)

# Consistent title case for descriptive text
for col in ["Customer_Name", "Product", "Category", "Brand", "Payment_Method",
            "Delivery_Status", "City", "Region", "Return_Status"]:
    df[col] = df[col].str.title()

# Convert numeric columns; invalid text becomes NaN
numeric_columns = ["Quantity", "Unit_Price", "Discount_Percent", "Shipping_Cost", "Rating"]
for col in numeric_columns:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# Parse dates safely
df["Order_Date"] = pd.to_datetime(df["Order_Date"], errors="coerce")
df["Ship_Date"] = pd.to_datetime(df["Ship_Date"], errors="coerce")

# Remove duplicate orders, keeping the first occurrence
df = df.drop_duplicates(subset="Order_ID", keep="first").copy()

# Fill selected missing values with defensible defaults/statistics
df["Customer_Name"] = df["Customer_Name"].fillna("Unknown Customer")
df["Payment_Method"] = df["Payment_Method"].fillna("Unknown")
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())
df["Unit_Price"] = df.groupby("Product")["Unit_Price"].transform(lambda s: s.fillna(s.media
df["Shipping_Cost"] = df["Shipping_Cost"].fillna(df["Shipping_Cost"].median())
df["Rating"] = df["Rating"].fillna(df["Rating"].median())
df["Region"] = df["Region"].fillna("Unknown")
df["City"] = df["City"].fillna("Unknown")

# Remove rows where essential transaction fields or dates are unavailable
df = df.dropna(subset=["Order_Date", "Quantity", "Unit_Price"]).copy()

# Standardize status labels
df["Delivery_Status"] = df["Delivery_Status"].replace({"Delivered": "Delivered"})
df["Return_Status"] = df["Return_Status"].fillna("No")

df.reset_index(drop=True, inplace=True)
df.head()
```

Out[9]:

	Order_ID	Customer_ID	Customer_Name	Product_ID	Product	Category	Brand	Quantity	Unit_Price
0	ORD1001	C201	Customer 201	P001	Laptop	Electronics	Aster	1.0	65000.0
1	ORD1002	C202	Customer 202	P002	Smartphone	Electronics	Nova	2.0	28000.0
2	ORD1003	C203	Customer 203	P003	Headphones	Accessories	Soundmax	3.0	2500.0
3	ORD1004	C204	Customer 204	P004	Smartwatch	Wearables	Pulse	1.0	6500.0
4	ORD1005	C205	Customer 205	P005	Tablet	Electronics	Aster	2.0	22000.0

Cleaning note: invalid numeric strings are converted to missing values. Missing numeric values are imputed using medians; rows with invalid order dates are removed because they cannot support time-based analysis. Duplicate Order_ID records are reduced to one order.

In [10]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 58 entries, 0 to 57
Data columns (total 19 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Order_ID          58 non-null     object
 1   Customer_ID       58 non-null     object
 2   Customer_Name     58 non-null     string
 3   Product_ID        58 non-null     object
 4   Product           58 non-null     string
 5   Category          58 non-null     string
 6   Brand             58 non-null     string
 7   Quantity          58 non-null     float64
 8   Unit_Price        58 non-null     float64
 9   Discount_Percent  57 non-null     float64
 10  Shipping_Cost     58 non-null     float64
 11  Payment_Method    58 non-null     string
 12  Order_Date        58 non-null     datetime64[ns]
 13  Ship_Date         58 non-null     datetime64[ns]
 14  Delivery_Status   58 non-null     string
 15  City              58 non-null     string
 16  Region            58 non-null     string
 17  Rating            58 non-null     float64
 18  Return_Status     58 non-null     string
dtypes: datetime64[ns](2), float64(5), object(3), string(9)
memory usage: 8.7+ KB
```

In [11]:
```
df.isnull().sum()
```

Out[11]:
```
Order_ID            0
Customer_ID         0
Customer_Name       0
Product_ID          0
Product             0
Category            0
Brand               0
Quantity            0
Unit_Price          0
Discount_Percent    1
Shipping_Cost       0
Payment_Method      0
Order_Date          0
Ship_Date           0
Delivery_Status     0
City                0
Region              0
Rating              0
Return_Status       0
dtype: int64
```

In [12]:
```
df.duplicated().sum(), df["Order_ID"].duplicated().sum()
```

Out[12]:
```
(0, 0)
```

In [13]:
```
df.describe()
```

Out[13]:

	Quantity	Unit_Price	Discount_Percent	Shipping_Cost	Rating
count	58.000000	58.000000	57.000000	58.000000	58.000000
mean	2.000000	15448.275862	5.350877	150.258621	3.982759
std	0.917663	18805.292661	4.397610	137.408414	0.927010
min	1.000000	700.000000	0.000000	30.000000	2.000000
25%	1.000000	1800.000000	0.000000	50.000000	3.250000
50%	2.000000	6500.000000	5.000000	87.500000	4.000000
75%	2.000000	26500.000000	10.000000	232.500000	5.000000
max	4.000000	65000.000000	12.000000	500.000000	5.000000

In [15]:
```
# Calculate discount amount, net item revenue, order total, shipping-inclusive sales
df["Gross_Amount"] = df["Quantity"] * df["Unit_Price"]
df["Discount_Amount"] = df["Gross_Amount"] * df["Discount_Percent"] / 100
df["Net_Revenue"] = df["Gross_Amount"] - df["Discount_Amount"]
df["Order_Total"] = df["Net_Revenue"] + df["Shipping_Cost"]

# Simple practice cost/profit assumption: cost is 70% of unit selling price.
# This is an illustrative assumption, not actual business cost data.
df["Estimated_Cost"] = df["Gross_Amount"] * 0.70
df["Estimated_Profit"] = df["Net_Revenue"] - df["Estimated_Cost"]

df["Order_Month"] = df["Order_Date"].dt.to_period("M")
df["Order_Day"] = df["Order_Date"].dt.day_name()
df["Shipping_Days"] = (df["Ship_Date"] - df["Order_Date"]).dt.days

df[["Order_ID","Product","Quantity","Unit_Price","Discount_Percent",
    "Gross_Amount","Discount_Amount","Net_Revenue","Order_Total","Estimated_Profit"]].head(
```

Out[15]:

	Order_ID	Product	Quantity	Unit_Price	Discount_Percent	Gross_Amount	Discount_Amount	Net_Revenue
0	ORD1001	Laptop	1.0	65000.0	5.0	65000.0	3250.0	61750.0
1	ORD1002	Smartphone	2.0	28000.0	10.0	56000.0	5600.0	50400.0
2	ORD1003	Headphones	3.0	2500.0	0.0	7500.0	0.0	7500.0
3	ORD1004	Smartwatch	1.0	6500.0	8.0	6500.0	520.0	5980.0
4	ORD1005	Tablet	2.0	22000.0	5.0	44000.0	2200.0	41800.0

In [16]:
```
print("Total orders:", df["Order_ID"].nunique())
print("Unique customers:", df["Customer_ID"].nunique())
print("Unique products:", df["Product"].nunique())
print("Units sold:", df["Quantity"].sum())
print("Gross sales: ₹", round(df["Gross_Amount"].sum(), 2))
print("Discount given: ₹", round(df["Discount_Amount"].sum(), 2))
print("Net revenue: ₹", round(df["Net_Revenue"].sum(), 2))
print("Order value including shipping: ₹", round(df["Order_Total"].sum(), 2))
print("Estimated profit: ₹", round(df["Estimated_Profit"].sum(), 2))
print("Average order value: ₹", round(df["Order_Total"].mean(), 2))
```

```
Total orders: 58
Unique customers: 25
Unique products: 12
Units sold: 116.0
Gross sales: ₹ 1290100.0
Discount given: ₹ 80160.0
Net revenue: ₹ 1202440.0
Order value including shipping: ₹ 1211095.0
Estimated profit: ₹ 304620.0
Average order value: ₹ 21247.28
```

In [17]:
```
product_summary = df.groupby("Product").agg(
    Orders=("Order_ID","nunique"), Units_Sold=("Quantity","sum"),
    Net_Revenue=("Net_Revenue","sum"), Average_Order_Revenue=("Net_Revenue","mean"),
    Average_Rating=("Rating","mean")
).sort_values("Net_Revenue", ascending=False)
product_summary
```

Out[17]:

Product	Orders	Units_Sold	Net_Revenue	Average_Order_Revenue	Average_Rating
Laptop	6	7.0	383260.0	63876.666667	4.5
Smartphone	5	10.0	252000.0	50400.000000	4.0
Tablet	5	10.0	209000.0	41800.000000	3.0
Camera	5	5.0	152000.0	30400.000000	5.0
Monitor	4	4.0	51040.0	12760.000000	2.0
Speaker	5	10.0	42000.0	8400.000000	4.0
Headphones	5	15.0	30000.0	7500.000000	4.0
Smartwatch	5	5.0	29900.0	5980.000000	5.0
Keyboard	5	10.0	18000.0	3600.000000	4.0
Charger	5	20.0	14000.0	2800.000000	3.0
Power Bank	4	8.0	11520.0	2880.000000	5.0
Mouse	4	12.0	9720.0	2430.000000	4.0

In [18]:
```
category_summary = df.groupby("Category").agg(
    Orders=("Order_ID","nunique"), Units=("Quantity","sum"),
    Revenue=("Net_Revenue","sum"), Estimated_Profit=("Estimated_Profit","sum")
).sort_values("Revenue", ascending=False)
category_summary
```

Out[18]:

Category	Orders	Units	Revenue	Estimated_Profit
Electronics	25	36.0	1047300.0	261550.0
Accessories	23	65.0	83240.0	23320.0
Audio	5	10.0	42000.0	12600.0
Wearables	5	5.0	29900.0	7150.0

In [19]:
```
region_summary = df.groupby(["Region","City"]).agg(
    Orders=("Order_ID","nunique"), Revenue=("Net_Revenue","sum"),
    Units=("Quantity","sum")
).sort_values("Revenue", ascending=False)
region_summary
```

Out[19]:

Region	City	Orders	Revenue	Units
South	**Hyderabad**	9	380220.0	18.0
	Chennai	10	315800.0	15.0
West	**Pune**	9	220200.0	26.0
South	**Bengaluru**	10	182000.0	20.0
West	**Mumbai**	10	71900.0	15.0
South	**Warangal**	9	29520.0	18.0
West	**Chennai**	1	2800.0	4.0

In [20]:
```
payment_summary = df.groupby("Payment_Method").agg(
    Orders=("Order_ID","nunique"), Revenue=("Order_Total","sum")
).sort_values("Orders", ascending=False)
payment_summary
```

Out[20]:

Payment_Method	Orders	Revenue
Upi	25	589200.0
Debit Card	10	72690.0
Credit Card	9	304790.0
Net Banking	9	221525.0
Cash On Delivery	4	9880.0
Unknown	1	13010.0

In [21]:
```
monthly_summary = df.groupby("Order_Month").agg(
    Orders=("Order_ID","nunique"), Units=("Quantity","sum"),
    Net_Revenue=("Net_Revenue","sum"), Average_Order_Value=("Order_Total","mean")
).sort_index()
monthly_summary
```

Out[21]:

Order_Month	Orders	Units	Net_Revenue	Average_Order_Value
2026-01	11	22.0	227820.0	20868.181818
2026-02	8	15.0	248420.0	31217.500000
2026-03	10	20.0	215510.0	21700.000000
2026-04	10	20.0	185300.0	18685.000000
2026-05	11	23.0	217220.0	21876.500000
2026-06	8	16.0	108170.0	13648.750000

In [22]:
```
customer_summary = df.groupby(["Customer_ID","Customer_Name"]).agg(
    Orders=("Order_ID","nunique"), Total_Spent=("Order_Total","sum"),
    Units=("Quantity","sum")
).sort_values("Total_Spent", ascending=False)
customer_summary.head(10)
```

Out[22]:

Customer_ID	Customer_Name	Orders	Total_Spent	Units
C213	**Customer 213**	2	174550.0	4.0
C201	**Customer 201**	3	112800.0	6.0
C225	**Customer 225**	2	112800.0	3.0
C212	**Customer 212**	2	65180.0	3.0
C224	**Customer 224**	2	65180.0	3.0
C202	**Customer 202**	3	64170.0	6.0
C214	**Customer 214**	2	58110.0	5.0
C203	**Customer 203**	3	55600.0	6.0
C208	**Customer 208**	3	52200.0	4.0
C204	**Customer 204**	3	51690.0	5.0

In [23]:
```
product_summary[product_summary["Net_Revenue"] > product_summary["Net_Revenue"].mean()]
```

Out[23]:

Product	Orders	Units_Sold	Net_Revenue	Average_Order_Revenue	Average_Rating
Laptop	6	7.0	383260.0	63876.666667	4.5
Smartphone	5	10.0	252000.0	50400.000000	4.0
Tablet	5	10.0	209000.0	41800.000000	3.0
Camera	5	5.0	152000.0	30400.000000	5.0

In [24]:
```
df["Delivery_Status"].value_counts()
```

Out[24]:
```
Delivered    38
Shipped      10
Cancelled     5
Returned      5
Name: Delivery_Status, dtype: Int64
```

In [25]:
```
df.groupby("Category").agg(
    Average_Discount=("Discount_Percent","mean"),
    Discount_Amount=("Discount_Amount","sum"),
    Net_Revenue=("Net_Revenue","sum")
).sort_values("Net_Revenue", ascending=False)
```

Out[25]:

Category	Average_Discount	Discount_Amount	Net_Revenue
Electronics	7.400000	75200.0	1047300.0
Accessories	3.636364	2360.0	83240.0
Audio	0.000000	0.0	42000.0
Wearables	8.000000	2600.0	29900.0

In [26]:
```
df[df["Discount_Percent"] >= 10][
    ["Order_ID","Product","Discount_Percent","Gross_Amount","Discount_Amount","Net_Revenue"
].sort_values("Discount_Amount", ascending=False)
```

Out[26]:

	Order_ID	Product	Discount_Percent	Gross_Amount	Discount_Amount	Net_Revenue
1	ORD1002	Smartphone	10.0	56000.0	5600.0	50400.0
24	ORD1026	Smartphone	10.0	56000.0	5600.0	50400.0
48	ORD1050	Smartphone	10.0	56000.0	5600.0	50400.0
13	ORD1014	Smartphone	10.0	56000.0	5600.0	50400.0
36	ORD1038	Smartphone	10.0	56000.0	5600.0	50400.0
42	ORD1044	Monitor	12.0	14500.0	1740.0	12760.0
30	ORD1032	Monitor	12.0	14500.0	1740.0	12760.0
18	ORD1020	Monitor	12.0	14500.0	1740.0	12760.0
7	ORD1008	Laptop	12.0	14500.0	1740.0	12760.0
54	ORD1056	Monitor	12.0	14500.0	1740.0	12760.0
22	ORD1024	Power Bank	10.0	3200.0	320.0	2880.0
34	ORD1036	Power Bank	10.0	3200.0	320.0	2880.0
11	ORD1012	Power Bank	10.0	3200.0	320.0	2880.0
46	ORD1048	Power Bank	10.0	3200.0	320.0	2880.0
29	ORD1031	Mouse	10.0	2700.0	270.0	2430.0
6	ORD1007	Mouse	10.0	2700.0	270.0	2430.0
41	ORD1043	Mouse	10.0	2700.0	270.0	2430.0
53	ORD1055	Mouse	10.0	2700.0	270.0	2430.0

In [27]:
```
df.groupby("Product").agg(
    Average_Rating=("Rating","mean"), Rating_Count=("Rating","count")
).sort_values("Average_Rating", ascending=False)
```

Out[27]:

Product	Average_Rating	Rating_Count
Camera	5.0	5
Power Bank	5.0	4
Smartwatch	5.0	5
Laptop	4.5	6
Headphones	4.0	5
Keyboard	4.0	5
Mouse	4.0	4
Smartphone	4.0	5
Speaker	4.0	5
Charger	3.0	5
Tablet	3.0	5
Monitor	2.0	4

In [28]:
```
customer_orders = df.groupby("Customer_ID")["Order_ID"].nunique()
repeat_customers = customer_orders[customer_orders > 1].sort_values(ascending=False)
repeat_customers
```

Out[28]:
```
Customer_ID
C201    3
C203    3
C204    3
C205    3
C206    3
C207    3
C208    3
C209    3
C202    3
C217    2
C224    2
C223    2
C222    2
C221    2
C220    2
C218    2
C213    2
C216    2
C215    2
C214    2
C212    2
C211    2
C210    2
C225    2
Name: Order_ID, dtype: int64
```

In [29]:
```
pd.crosstab(df["Payment_Method"], df["Delivery_Status"])
```

Out[29]:

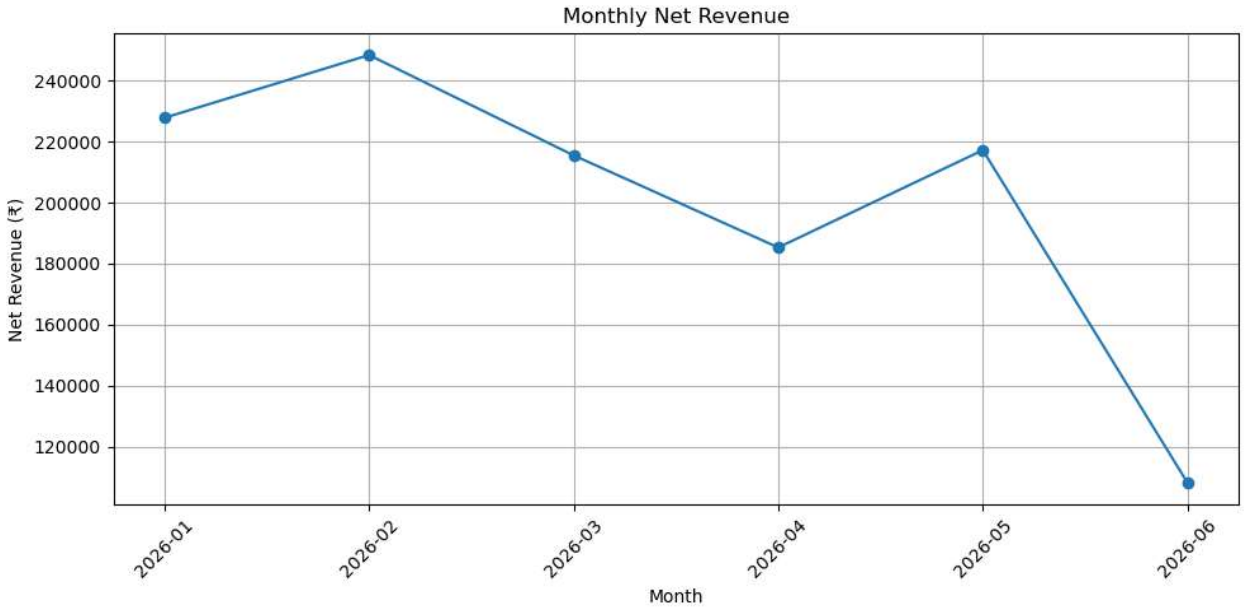
Payment_Method \ Delivery_Status	Cancelled	Delivered	Returned	Shipped
Cash On Delivery	0	4	0	0
Credit Card	0	5	4	0
Debit Card	0	10	0	0
Net Banking	0	9	0	0
Unknown	0	0	1	0
Upi	5	10	0	10

In [30]:
```
df[["Quantity","Unit_Price","Discount_Percent","Shipping_Cost",
    "Gross_Amount","Net_Revenue","Order_Total","Estimated_Profit","Rating"]].corr()
```

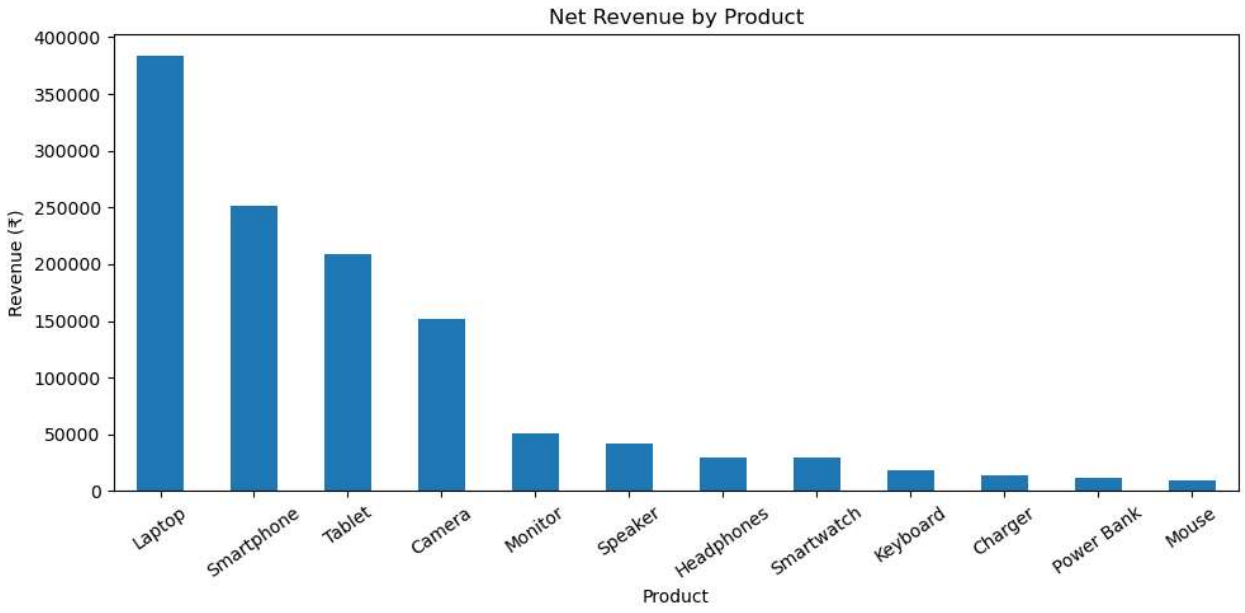
Out[30]:

	Quantity	Unit_Price	Discount_Percent	Shipping_Cost	Gross_Amount	Net_Revenue	Order_T
Quantity	1.000000	-0.510245	-0.441769	-0.610792	-0.333753	-0.324719	-0.326
Unit_Price	-0.510245	1.000000	0.177109	0.943471	0.901084	0.904033	0.905
Discount_Percent	-0.441769	0.177109	1.000000	0.206220	0.181082	0.160859	0.161
Shipping_Cost	-0.610792	0.943471	0.206220	1.000000	0.773533	0.778173	0.780
Gross_Amount	-0.333753	0.901084	0.181082	0.773533	1.000000	0.999454	0.999
Net_Revenue	-0.324719	0.904033	0.160859	0.778173	0.999454	1.000000	0.999
Order_Total	-0.326725	0.905385	0.161318	0.780404	0.999415	0.999994	1.000
Estimated_Profit	-0.315923	0.908722	0.100718	0.791201	0.991586	0.995322	0.995
Rating	-0.288726	0.282639	-0.041894	0.178396	0.175645	0.184580	0.184

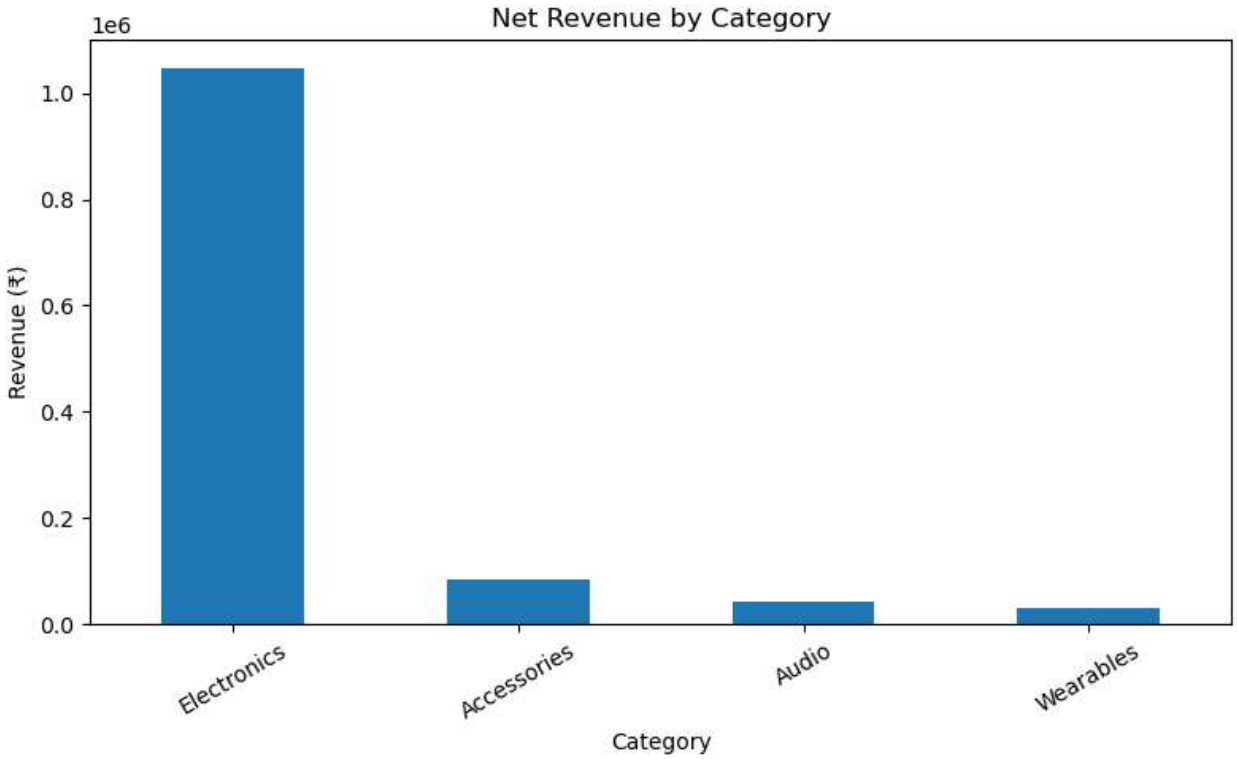
In [31]:
```
monthly_revenue = df.groupby("Order_Month")["Net_Revenue"].sum()
plt.figure(figsize=(10,5))
plt.plot(monthly_revenue.index.astype(str), monthly_revenue.values, marker="o")
plt.title("Monthly Net Revenue")
plt.xlabel("Month"); plt.ylabel("Net Revenue (₹)")
plt.xticks(rotation=45); plt.grid(True); plt.tight_layout(); plt.show()
```



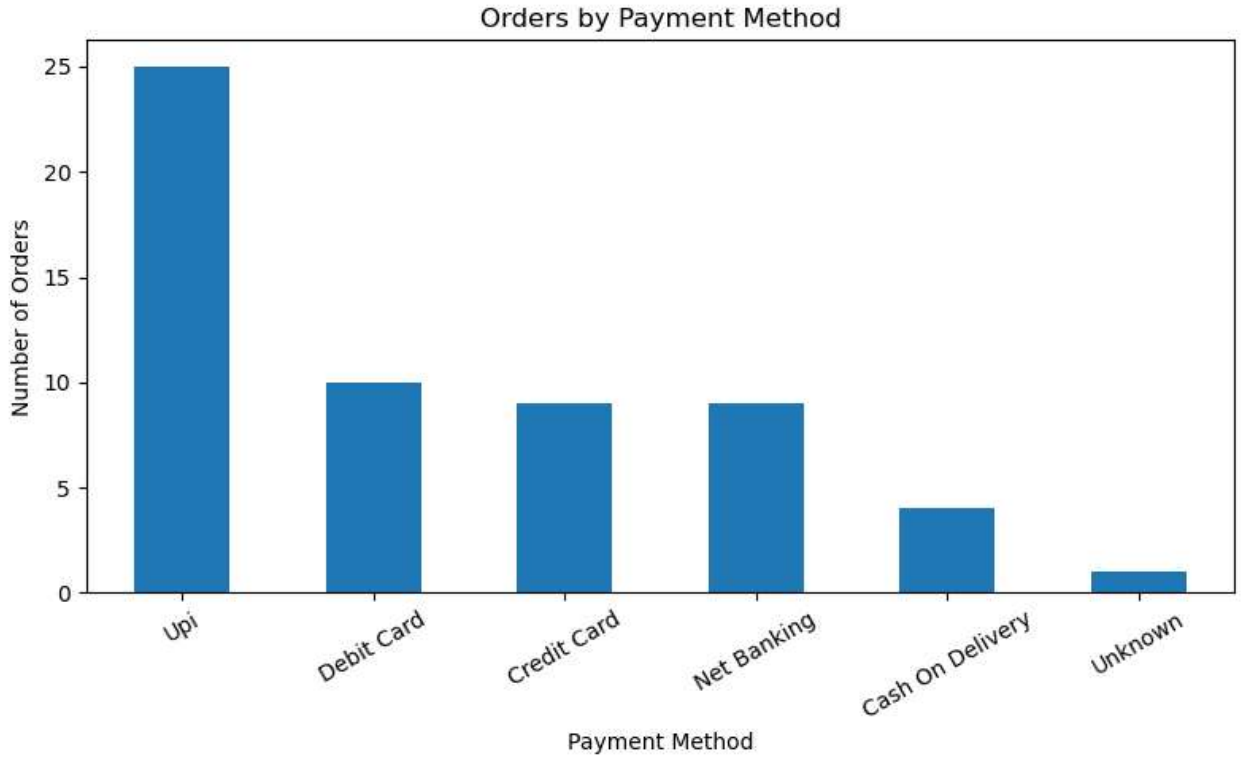
In [32]:
```
product_revenue = df.groupby("Product")["Net_Revenue"].sum().sort_values(ascending=False)
plt.figure(figsize=(10,5))
product_revenue.plot(kind="bar")
plt.title("Net Revenue by Product"); plt.xlabel("Product"); plt.ylabel("Revenue (₹)")
plt.xticks(rotation=35); plt.tight_layout(); plt.show()
```



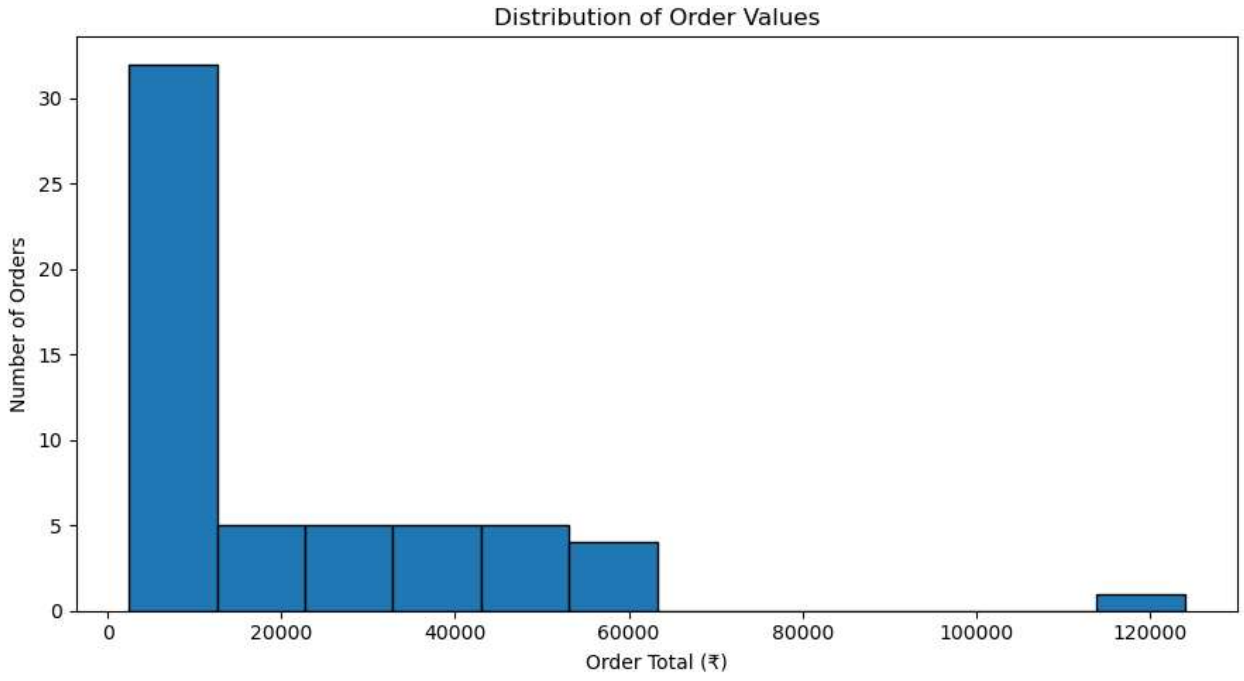
In [33]:
```
category_revenue = df.groupby("Category")["Net_Revenue"].sum().sort_values(ascending=False)
plt.figure(figsize=(8,5))
category_revenue.plot(kind="bar")
plt.title("Net Revenue by Category"); plt.xlabel("Category"); plt.ylabel("Revenue (₹)")
plt.xticks(rotation=30); plt.tight_layout(); plt.show()
```



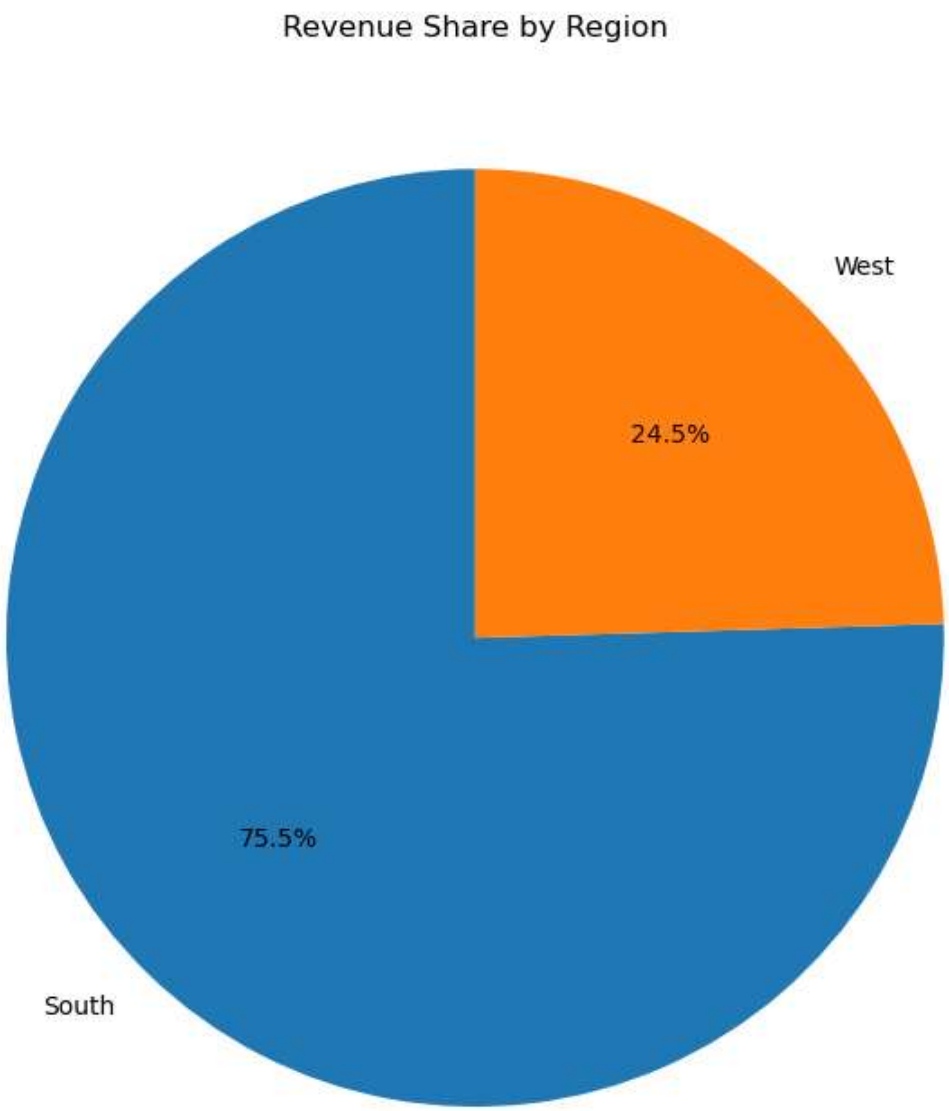
In [34]:
```
payment_counts = df["Payment_Method"].value_counts()
plt.figure(figsize=(8,5))
payment_counts.plot(kind="bar")
plt.title("Orders by Payment Method"); plt.xlabel("Payment Method"); plt.ylabel("Number of
plt.xticks(rotation=30); plt.tight_layout(); plt.show()
```



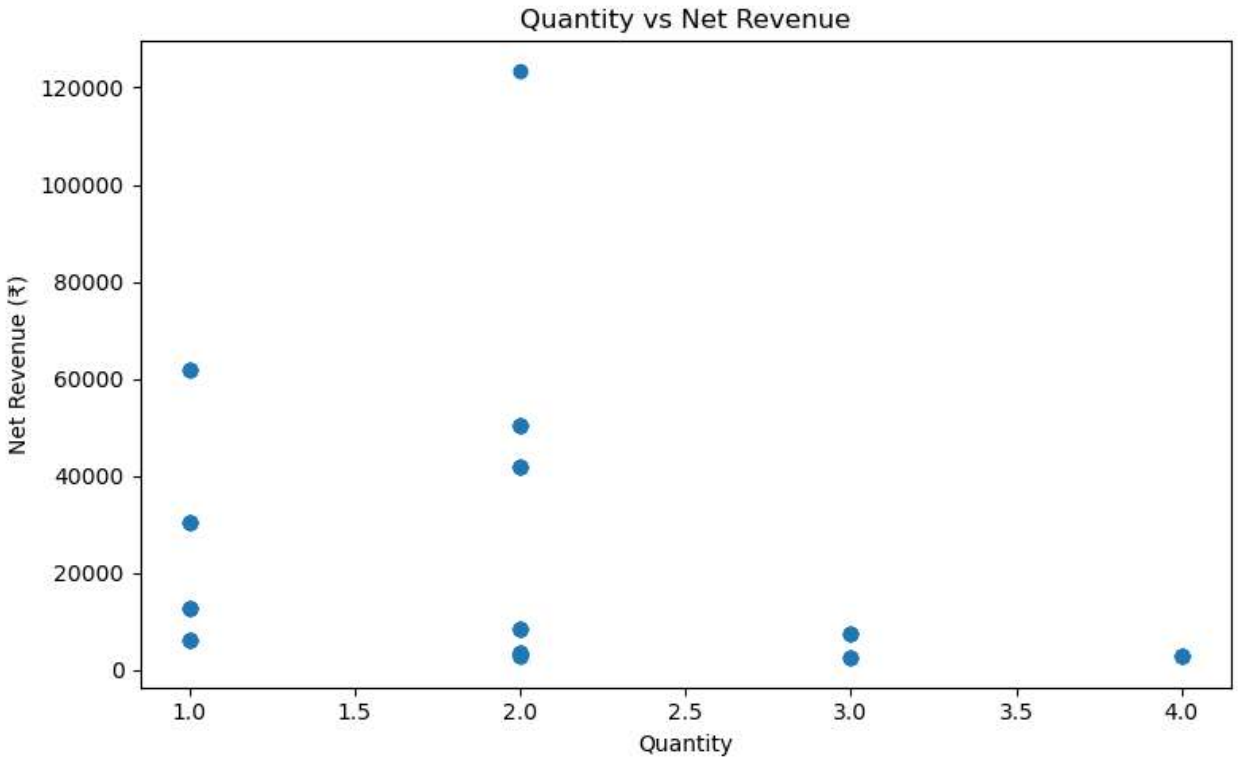
In [35]:
```
plt.figure(figsize=(9,5))
plt.hist(df["Order_Total"], bins=12, edgecolor="black")
plt.title("Distribution of Order Values"); plt.xlabel("Order Total (₹)"); plt.ylabel("Numbe
plt.tight_layout(); plt.show()
```



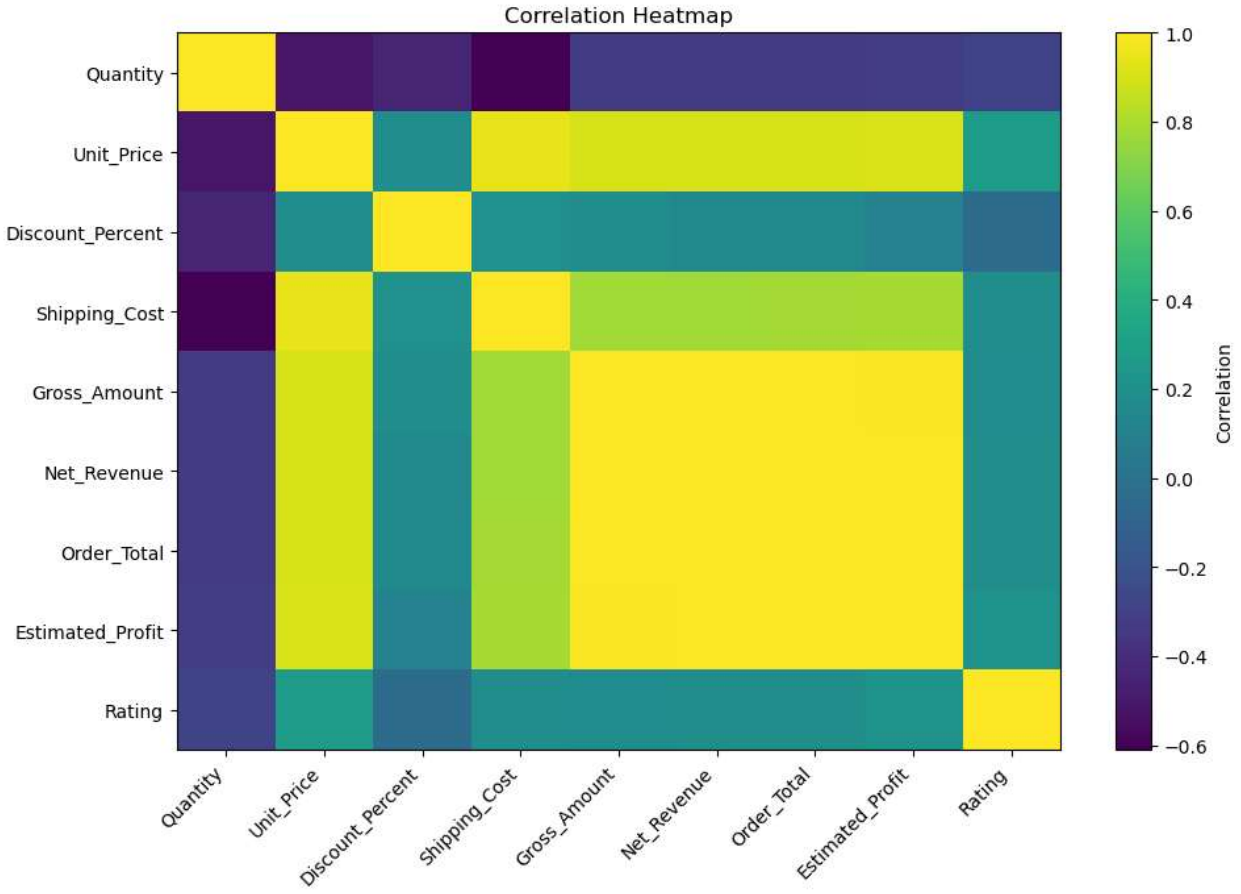
In [36]:
```
region_revenue = df.groupby("Region")["Net_Revenue"].sum()
plt.figure(figsize=(7,7))
plt.pie(region_revenue, labels=region_revenue.index, autopct="%1.1f%%", startangle=90)
plt.title("Revenue Share by Region"); plt.tight_layout(); plt.show()
```



In [37]:
```
plt.figure(figsize=(8,5))
plt.scatter(df["Quantity"], df["Net_Revenue"])
plt.title("Quantity vs Net Revenue"); plt.xlabel("Quantity"); plt.ylabel("Net Revenue (₹)")
plt.tight_layout(); plt.show()
```



In [38]:
```
corr = df[["Quantity","Unit_Price","Discount_Percent","Shipping_Cost",
           "Gross_Amount","Net_Revenue","Order_Total","Estimated_Profit","Rating"]].corr()
plt.figure(figsize=(10,7))
plt.imshow(corr, aspect="auto")
plt.colorbar(label="Correlation")
plt.xticks(range(len(corr.columns)), corr.columns, rotation=45, ha="right")
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title("Correlation Heatmap")
plt.tight_layout(); plt.show()
```



In [ ]: